

MS MARCO Generative QA — Progress Report

Generated 2026-05-14 17:43. For per-week depth see reports/generated/weekNN_.pdf.*

Project goal

Build an end-to-end open-domain QA system on MS MARCO that goes **query** → **retrieve passages** → **generate an answer**, and measure each stage in isolation so it's clear where errors come from. The aim is not SOTA; the aim is a clean, reproducible pipeline whose retrieval, reranking, and generation stages can each be ablated.

Methods (by week)

Week	Stage	Data	Method
W1	EDA	MS MARCO QA v2.1 validation, 5k-row sample	Length / type / answer-type distributions; sizing decisions for W2–W5
W2	Sparse retrieval (BM25)	MS MARCO Passage (8.8M docs), dev/small (6 980 q)	bm25s backend, $k1=1.5$, $b=0.75$, top-1000 per query
W3	RAG generation	BM25 / reranked top-3 $\cap$ QA references, full dev/small (6 980 q)	T5-small (no fine-tuning), prompt = question: <q> context: <p1 p2 p3>
W4	Dense retrieval	Qrels-anchored 50k sample of Passage, dev/small (6 980 q)	all-MiniLM-L6-v2 + FAISS IndexFlatIP; same sample re-indexed with BM25 for apples-to-apples comparison
W5	Cross-encoder reranking	W4 dense top-100, full dev/small (6 980 q)	cross-encoder/ms-marco-MiniLM-L-6-v2 rerank, depth K=100

All runs share a single config (configs/baseline.yaml); the four entrypoints in experiments/run_*.py produce the metrics each report cites.

Key results

W1 (EDA) — Queries are short (5–10 words median); passages ~10× longer (median ~50 words). DESCRIPTION and NUMERIC dominate the query-type buckets; a non-trivial fraction of queries has no answer and must be filtered for any SFT supervision. Most queries have 0 or 1 marked-relevant passage (sparse, binary qrels).

W2 (BM25 baseline, full 8.8M corpus, 6 980 dev/small queries)

Metric	Value
MRR@10	0.1703
Recall@100	0.6212
Recall@1000	0.8154

Published Anserini/Lucene reference is ~0.184 MRR@10; the ~0.014 gap is attributable to tokenizer differences between Lucene and bm25s. No k1/b tuning. Search cost: 586 ms/query.

W3 (RAG, T5-small + retrieved top-3, full dev/small 6 980 queries)

Two paired runs over the **same 6 980 dev/small qids** (mutually restricted via `--restrict-to-run`), differing only in upstream retrieval source:

Retrieval → T5-small	ROUGE-L	BLEU	EM	Token-F1
BM25 top-3	0.1859	0.0717	0.0135	0.1966
Reranked top-3	0.3621	0.2922	0.0606	0.3677
Δ (rerank – BM25)	+0.1763	+0.2206	+0.0471	+0.1711

Reranking the first stage **roughly doubles every generation metric** on the full benchmark. 95 % paired-bootstrap CIs (n = 6 980, 10 k resamples, seed 42) on the per-query Δ all sit strictly above zero: ROUGE-L [+0.1663, +0.1820], BLEU [+0.1265, +0.1395], EM [+0.0417, +0.0527], Token-F1 [+0.1632, +0.1789]; two-sided p < 0.001 in every case.

Mechanism (retrieval flag): the rate of having at least one relevance-judged passage in the top-3 jumps from **20.8 % (BM25) to 96.9 % (reranked)** — the cross-encoder is what gets the relevant passage into the generator's window.

By query type (token-F1 Δ), DESCRIPTION (n=3725) gains most (+0.2050); NUMERIC (n=1665) gains least (+0.1238) — short numeric answers depend more on lexical surface match than on which-of-the- near-duplicates the reranker picks. Per-query split: **4 015 improvements / 1 766 regressions / 1 199 ties** over 6 980 queries.

T5-small is pretrained-only (no SFT) so absolute level numbers are modest. The load-bearing claim is the **paired Δ between rows**, which is statistically robust on n = 6 980.

A low-cost semantic evaluation sanity check on 3 000 paired examples using DistilBERT-based BERTScore (rescaled, paired-bootstrap CI; seed 42) gives BM25 0.2192 → reranked 0.3920, Δ **+0.1728** ([+0.1608, +0.1850], p < 0.001). Per-query:

rerank strictly better on 64.8 % of qids; tie 5.7 %; strictly worse on 29.5 %. The semantic-proxy Δ (+0.1728) is within a hair of the surface-form Token-F1 Δ (+0.1711) and ROUGE-L Δ (+0.1742), so the reranker improvement **is not a surface-form artefact** — it shows up in a semantic-similarity scorer at the same magnitude. (DistilBERT is intentionally not the canonical BERTScore encoder; this is a proxy that complements rather than replaces a full roberta-large evaluation.)

A 40-query seeded triage of the 233-strong regression bucket shows that **~90 % of regressions are generation-side truncation, not retrieval or semantic failures**: 55 % truncation_midword (output cut at max_new_tokens = 64), 35 % truncation_short (≤ 3 tokens, generator extracted only a title-like fragment), 5 % topic_drift, 5 % extractive_passage_bias, 0 % semantic_mismatch. Cheapest immediate mitigation: raise the generator's token budget and re-evaluate before any heavier intervention.

W4 (Dense vs BM25 on the same 50k qrels-anchored sample, 6 980 queries)

Metric	BM25 (sample)	Dense (sample)	Δ
MRR@10	0.6948	0.8830	+0.1882
nDCG@10	0.7270	0.9041	+0.1771
Recall@100	0.9338	0.9946	+0.0608
Recall@1000	0.9686	0.9991	+0.0305

Dense wins on every metric. Caveat: every dev/small relevant doc is unconditionally included in the 50k pool, so absolute numbers are upper-bounded; the *delta* between dense and BM25 on the same pool is what's meaningful. Recall@100 = 0.9946 means dense has effectively hit the ceiling on this sampled setting → from W5 onward, recall is no longer the bottleneck.

W5 (Cross-encoder rerank over W4 dense top-100, full 6 980-query dev/small)

Metric	Dense (top-100)	Dense + CE rerank	Δ
MRR@10	0.8830	0.9304	+0.0474
nDCG@10	0.9041	0.9434	+0.0393
Recall@100	0.9946	0.9946	+0.0000

Recall@100 is unchanged by construction (the reranker only re-orders the top-100). The interesting columns are MRR@10 and nDCG@10: even with recall saturated, the cross-encoder still buys roughly +0.04 by tightening *local ordering* — pushing the relevant passage from rank 2–3 to rank 1. Cost: 4 h 37 min wall-clock for 538 k pairs (~32 pairs/s), peak RSS 3.3 GiB; the CE is the slowest stage by ~2 orders of magnitude. The earlier 1 000-query subsample of dev/small produced consistent deltas (MRR Δ +0.0435, nDCG Δ +0.0398), confirming the gain is not an artefact of the subsample.

Limitations

- **Sampled corpus from W4 onward.** Dense + rerank numbers are on a 50k qrels-anchored sample where every relevant doc is present by construction.

Comparable *within* W4/W5; the W4/W5 absolute MRR is *not* directly comparable to the W2 full-corpus number.

- **No fine-tuning anywhere.** Generic encoders/generator used as-is (all-MiniLM-L6-v2, ms-marco-MiniLM-L-6-v2, t5-small). A MS-MARCO-tuned encoder (msmarco-MiniLM-L6-cos-v5) or SFT generator would close most of the gap to leaderboard baselines.
- **Dev/small only, single seed.** No held-out test split; distractor sample is deterministic but not averaged across seeds.
- **No fine-tuning of T5-small.** W3 absolute level numbers (ROUGE-L ~0.18 BM25 → ~0.36 reranked) are modest because the generator is pretrained-only; the Δ between paired rows is the load-bearing signal. A SFT pass on (question, gold passage, answer) triples is in scope for a future week.
- **Surface-form generation metrics only.** ROUGE/BLEU/EM penalise valid paraphrases — a semantic metric (BERTScore) would give a fairer signal.

Next steps

- **Bump T5-small max_new_tokens from 64 → 128** and re-run the W3 paired comparison. Regression taxonomy says 90 % of the 233 W3 regressions are truncation, not retrieval failure — this is the cheapest intervention with the highest expected payoff.
- Full-dev BERTScore with the canonical roberta-large encoder (the current 3 000-paired DistilBERT proxy is a sanity check, not the citation-grade evaluation).
- Rerank BM25 top-100 (W2 run) and compare the delta to the dense delta — does the reranker recover more from a weaker first stage?
- Hybrid first-stage (RRF over BM25 + dense) → rerank.
- Scale the dense sample from 50k to 200k–500k passages to track how the BM25[?]dense gap shifts as relevant-doc density drops.
- T5-small SFT on (question, gold passage, answer) triples — only pursue after the cheap interventions above, since the regression taxonomy already shows the bottleneck isn't generation quality but output truncation.

How this maps to the repo

- Conclusions per week: reports/generated/weekNN_*.pdf
- Process logs per week: notebooks/weekNN_*.ipynb
- Reproducible pipeline: experiments/run_*.py + src/ + configs/baseline.yaml
- Reproduce locally: make targets (see Makefile); outputs land in outputs/ (not committed).